ПРАКТИЧЕСКАЯ РАБОТА № 5

Нагрузочное тестирование Auction API в Apache JMeter

1 ЦЕЛЬ РАБОТЫ

Освоить нагрузочное тестирование веб-приложения в Apache JMeter. Требуется запустить сервер аукциона на Python (FastAPI + SQLite), создать тестовый план с группой из 5 пользователей, ramp-up 10 секунд и одним циклом, выполнить POST- и GET-запросы к /items/ и проанализировать результаты в слушателях JMeter.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Apache JMeter — инструмент для тестирования производительности и нагрузочного тестирования веб-приложений. Он имитирует заданное число пользователей, измеряет время отклика и пропускную способность, позволяет выявлять узкие места. Тестовый план состоит из иерархии элементов; основные из них приведены в таблице.

Таблица 1 – Компоненты JMeter и их применение

Компонент	Назначение	Применение в работе
Thread Group	Задаёт число виртуальных пользователей, ramp-up и число циклов.	5 пользователей, ramp-up 10 с, loop count 1.
Sampler (HTTP Request)	Отправляет запрос на сервер.	POST /items/ и GET /items/ к 127.0.0.1:8000.
Config Element	Настраивает общие параметры запросов.	HTTP Header Manager с `Content-Type: application/json`.
Assertions	Проверяют, что ответ соответствует ожиданиям.	Код ответа 200 и наличие поля `name` в JSON.
Listeners	Собирают и отображают результаты.	View Results Tree, Summary Report, Aggregate Report.
Non-GUI режим	Запуск без интерфейса для воспроизводимых замеров.	`jmeter -n -t ... -l results.jtl -e -o report`.

Ключевые метрики: **Samples** — число выполненных запросов; **Average** — среднее время отклика; **Min/Max** — минимальное и максимальное время; **90% Line** — значение, ниже которого уложились 90 % запросов; **Error %** — доля ошибочных ответов; **Throughput** — число запросов в секунду. Для оценки удовлетворённости пользователей JMeter рассчитывает APDEX.

3 СЕРВЕР АУКЦИОНА НА FASTAPI

Сервер реализует три эндпоинта: создание лота (`POST /items/`), получение списка лотов (`GET /items/`) и получение лота по идентификатору (`GET /items/{item_id}`). Дополнительно добавлен `GET /health` для быстрой проверки доступности. Данные хранятся в SQLite-файле `auction.db` рядом с `main.py`.

```
"""ПР5. Сервер аукциона на FastAPI для нагрузочного тестирования в JMeter.

Запуск:
    python -m uvicorn main:app --reload
Сервер доступен по адресу http://127.0.0.1:8000, документация — /docs.
"""

import sqlite3
from pathlib import Path
from typing import List, Optional

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

DB_PATH = Path(__file__).resolve().parent / "auction.db"

app = FastAPI(title="Auction API", version="1.0")


# ---------------------------------------------------------------------------
# База данных
# ---------------------------------------------------------------------------
def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS items (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            description TEXT,
            price REAL NOT NULL
        )
    """)
    conn.commit()
    conn.close()


init_db()


class Item(BaseModel):
    id: Optional[int] = None
    name: str
    description: Optional[str] = None
    price: float


# ---------------------------------------------------------------------------
# Эндпоинты
# ---------------------------------------------------------------------------
@app.post("/items/", response_model=Item)
def create_item(item: Item):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO items (name, description, price)
        VALUES (?, ?, ?)
        """,
```

```
        (item.name, item.description, item.price),
    )
    conn.commit()
    item_id = cursor.lastrowid
    conn.close()
    return Item(id=item_id, name=item.name, description=item.description,
price=item.price)


@app.get("/items/", response_model=List[Item])
def get_items():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM items")
    rows = cursor.fetchall()
    conn.close()
    return [
        Item(id=row[0], name=row[1], description=row[2], price=row[3])
        for row in rows
    ]


@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM items WHERE id = ?", (item_id,))
    row = cursor.fetchone()
    conn.close()
    if row is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return Item(id=row[0], name=row[1], description=row[2], price=row[3])


@app.get("/health")
def health():
    return {"status": "ok"}
```

Отличие от кода из задания: в модель `Item` добавлено необязательное поле `id`, чтобы POST возвращал идентификатор созданной записи; путь к базе строится от каталога файла, а не от текущего каталога процесса; запросы выполняются с параметрами `?`, что исключает SQL-инъекции. Логика и структура эндпоинтов при этом соответствуют заданию.

3.1 Запуск сервера

```
pip install "fastapi[all]" uvicorn
python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Модуль `sqlite3` входит в стандартную библиотеку Python, отдельная установка не требуется. После запуска сервер доступен по адресу `http://127.0.0.1:8000`, интерактивная документация — по адресу `/docs`. Проверка доступности выполнялась запросом к `/health`, который возвращает `{"status": "ok"}`.

4 ТЕСТОВЫЙ ПЛАН JMETER

План сохранён в файле `auction_test_plan.jmx`. Структура плана:

```
Auction API Load Test
├── Users: 5, Ramp-up: 10, Loop: 1        (Thread Group)
├── HTTP Header Manager                   Content-Type: application/json
├── Create Item (POST /items/)            Response Assertion: code = 200
│                                         JSONPath: $[0].name ~ Item.*  (для GET)
├── Get Items (GET /items/)               Response Assertion: code = 200
│                                         JSONPath: $[0].name ~ Item.*
├── View Results Tree
├── Summary Report
└── Aggregate Report
```

4.1 Группа потоков

Число потоков — 5, период разгона — 10 секунд, число циклов — 1. При таких настройках каждый пользователь выполняет один POST и один GET, итого 10 запросов. Разгон в 10 секунд распределяет старт пользователей и позволяет наблюдать рост нагрузки, а не одновременный залп.

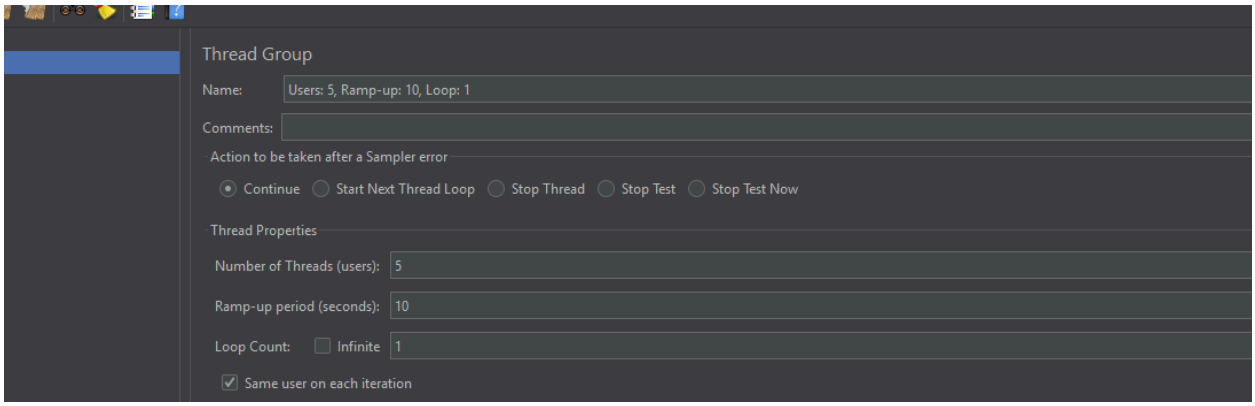


Рисунок 1 – Настройки Thread Group: 5 пользователей, ramp-up 10 с, 1 цикл

4.2 HTTP-запросы

POST-запрос отправляет JSON с уникальным именем лота: переменная `${__threadNum}` подставляет номер потока, поэтому пять пользователей создают пять разных записей. GET-запрос читает список лотов.

```
POST http://127.0.0.1:8000/items/
Content-Type: application/json

{
  "name": "Item ${__threadNum}",
  "description": "A description of Item ${__threadNum}",
  "price": 100.0
}

GET http://127.0.0.1:8000/items/
```

Для каждого запроса добавлены две проверки: Response Assertion на код 200 и JSONPath Assertion. Для GET проверяется, что первый элемент массива содержит поле `name`, соответствующее шаблону `Item.*`; это подтверждает, что ответ действительно является списком созданных лотов, а не пустым массивом или страницей ошибки.

4.3 Слушатели

View Results Tree показывает каждый запрос, его заголовки, тело и ответ; Summary Report даёт сводку по среднему, минимуму и максимуму; Aggregate Report добавляет медиану и перцентили. Все три слушателя использовались при отладке плана, а итоговые цифры дополнительно сняты в non-GUI режиме с формированием HTML-отчёта.

5 ВЫПОЛНЕНИЕ ТЕСТА

Основной прогон выполнен из командной строки:

```
jmeter -n -t auction_test_plan.jmx -l results.jtl -e -o report
```

Ключи: `-n` — non-GUI режим, `-t` — файл плана, `-l` — файл результатов в формате JTL, `-e` — формирование HTML-отчёта, `-o` — каталог отчёта. Non-GUI режим даёт более стабильные замеры, поскольку интерфейс JMeter не потребляет ресурсы во время теста.

Дополнительно план был открыт в графическом интерфейсе и запущен кнопкой Start — для подтверждения работоспособности плана и получения снимков экрана. Результаты обоих прогонов совпадают по числу запросов и отсутствию ошибок.

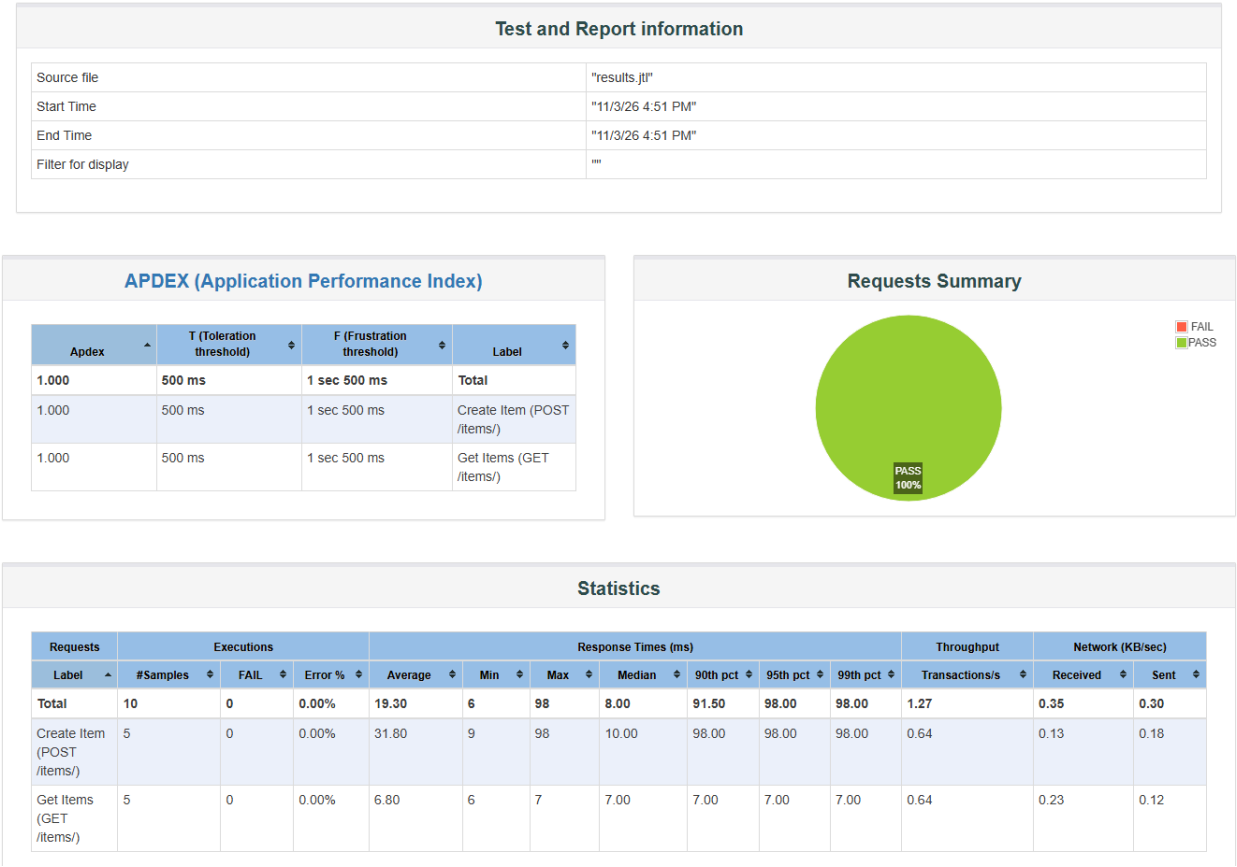
6 РЕЗУЛЬТАТЫ

Таблица 2 – Результаты

Запрос	Samples	Average, мс	Min, мс	Max, мс	90% Line, мс	Error %
Create Item (POST /items/)	5	31,8	9	98	98,0	0,00
Get Items (GET /items/)	5	6,8	6	7	7,0	0,00
Total	**10**	**19,3**	**6**	**98**	**91,5**	**0,00**

Все 10 запросов завершились успешно: доля ошибок 0,00 %, assertion'ы не сработали ни разу. Пропускная способность — 1,27 запроса в секунду, что

соответствует разгону 5 пользователей за 10 секунд (каждый пользователь делает два запроса). APDEX = 1,000. Медиана времени отклика — 8 мс.



Test and Report information

Source file	"results.jtl"
Start Time	"11/3/26 4:51 PM"
End Time	"11/3/26 4:51 PM"
Filter for display	""

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	**500 ms**	**1 sec 500 ms**	**Total**
1.000	500 ms	1 sec 500 ms	Create Item (POST /items/)
1.000	500 ms	1 sec 500 ms	Get Items (GET /items/)

Requests Summary

Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**10**	**0**	**0.00%**	**19.30**	**6**	**98**	**8.00**	**91.50**	**98.00**	**98.00**	**1.27**	**0.35**	**0.30**
Create Item (POST /items/)	5	0	0.00%	31.80	9	98	10.00	98.00	98.00	98.00	0.64	0.13	0.18
Get Items (GET /items/)	5	0	0.00%	6.80	6	7	7.00	7.00	7.00	7.00	0.64	0.23	0.12

Рисунок 2 – HTML-отчёт JMeter: APDEX 1,000, 10 запросов, 0 ошибок, среднее 19,3 мс

6.1 Слушатели в графическом интерфейсе

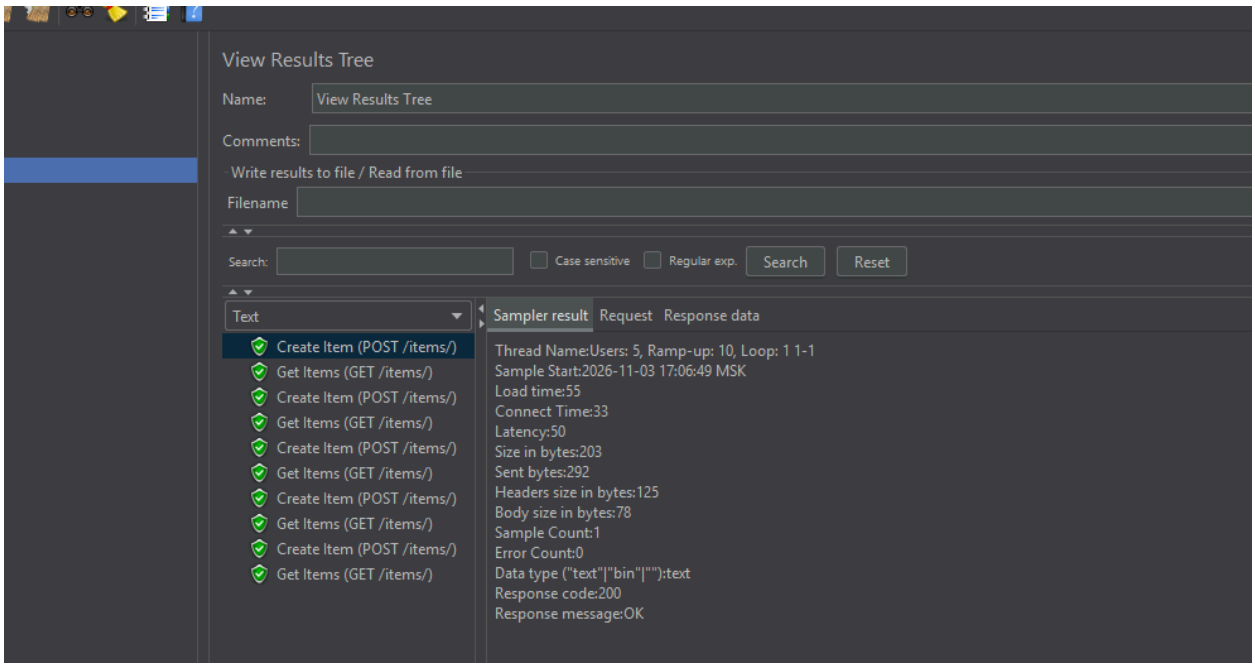


Рисунок 3 – View Results Tree: все запросы успешны, код ответа 200

View Results Tree подтверждает, что каждый POST и GET вернул код 200 и корректный JSON. В дереве видно по пять зелёных записей каждого типа — по числу потоков.

Summary Report и Aggregate Report показывают сводные метрики. На снимках GUI-прогона средние значения равны 38 и 4 мс; они незначительно отличаются от таблицы выше, поскольку снимки сделаны при повторном запуске плана. Основные цифры отчёта взяты из файла `results.jtl` и HTML-дашборда — они соответствуют одному прогону.

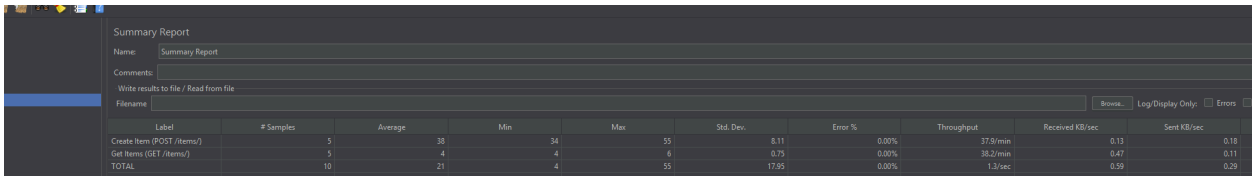


Рисунок 4 – Summary Report: 10 запросов, 0,00 % ошибок

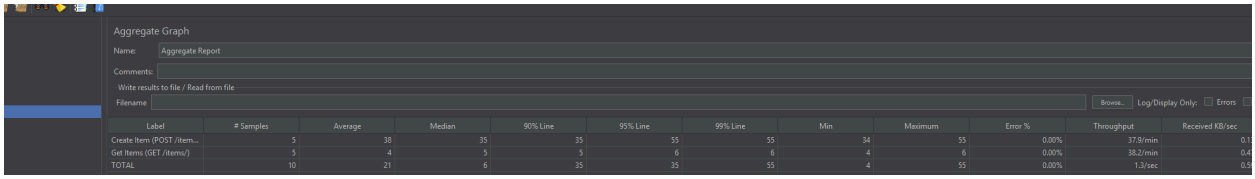


Рисунок 5 – Aggregate Report: медиана, 90/95/99-й перцентили, пропускная способность

7 АНАЛИЗ РЕЗУЛЬТАТОВ

GET-запросы стабильны и быстры: 6–7 мс, разброс практически отсутствует. POST-запросы медленнее в среднем (31,8 мс) и имеют выброс до 98 мс. Причина — операция записи в SQLite с `commit`: каждая транзакция сопровождается сбросом на диск, а первый запрос дополнительно оплачивает установку соединения и «холодный» старт. Для учебного приложения это ожидаемо; в продуктивной системе узким местом стали бы блокировки записи SQLite при одновременных INSERT.

Ошибок нет, поэтому по критерию «доля успешных запросов» система проходит тест. По времени отклика все запросы укладываются в сотни миллисекунд, что для локального сервера и пяти пользователей является хорошим результатом. Нагрузка в 5 пользователей не является стрессовой: для поиска предела следует увеличивать число потоков и строить график зависимости времени отклика от нагрузки.

Таблица 3 – Анализ результатов

Наблюдение	Причина	Рекомендация
POST в 4–5 раз медленнее GET	Запись в SQLite и commit на каждый запрос	Батчинг, WAL-режим или пул соединений
Единичный выброс 98 мс	Первый запрос: установка соединения и прогрев	Прогревочный запрос перед измерениями
Throughput 1,27 запроса/с	Ramp-up 5 пользователей за 10 с, по 2 запроса	Для замера предела увеличить число потоков и циклов
0 % ошибок, APDEX 1,000	Корректная работа сервера и плана	Использовать как базовую линию при изменениях

8 ВЫВОД

Разработан и выполнен тестовый план Apache JMeter для FastAPI-сервера аукциона. План содержит группу из 5 пользователей с ramp-up 10 секунд и одним циклом, POST- и GET-запросы к `/items/`, HTTP Header Manager, проверки кода ответа и содержимого JSON, а также три слушателя. В non-GUI режиме выполнено 10 запросов, все успешно: 0 ошибок, среднее время отклика 19,3 мс, медиана 8 мс, максимум 98 мс, пропускная способность 1,27 запроса в секунду, APDEX 1,000. POST-запросы ожидаемо медленнее GET из-за записи в SQLite; для повышения производительности предложены WAL-режим, пул соединений и прогревочный запрос.

ПРИЛОЖЕНИЕ А. КЛЮЧЕВОЙ ФРАГМЕНТ ТЕСТОВОГО ПЛАНА

Ниже приведены фрагменты `auction_test_plan.jmx`, задающие группу потоков, POST-запрос и проверки. Полный план приложен отдельным файлом.

```
<ThreadGroup guiclass="ThreadGroupGui" testclass="ThreadGroup"
             testname="Users: 5, Ramp-up: 10, Loop: 1" enabled="true">
  <stringProp name="ThreadGroup.on_sample_error">continue</stringProp>
  <stringProp name="ThreadGroup.num_threads">5</stringProp>
  <stringProp name="ThreadGroup.ramp_time">10</stringProp>
  <boolProp name="ThreadGroup.scheduler">false</boolProp>
</ThreadGroup>

<HTTPSamplerProxy testname="Create Item (POST /items/)" enabled="true">
  <stringProp name="HTTPSampler.domain">127.0.0.1</stringProp>
  <stringProp name="HTTPSampler.port">8000</stringProp>
  <stringProp name="HTTPSampler.path">/items/</stringProp>
  <stringProp name="HTTPSampler.method">POST</stringProp>
  <stringProp name="HTTPSampler.contentEncoding">UTF-8</stringProp>
  <boolProp name="HTTPSampler.postBodyRaw">true</boolProp>
  <!-- тело: {"name": "Item ${__threadNum}", "description": "...", "price": 100.0} -->
  <ResponseAssertion testname="Response code = 200">
    <stringProp name="Assertion.test_field">Assertion.response_code</stringProp>
    <stringProp name="Assertion.test_string">200</stringProp>
  </ResponseAssertion>
</HTTPSamplerProxy>

<JSONPathAssertion testname="JSON contains Item field">
  <stringProp name="JSON_PATH">$[0].name</stringProp>
  <stringProp name="EXPECTED_VALUE">Item.*</stringProp>
  <boolProp name="ISREGEX">true</boolProp>
</JSONPathAssertion>
```